

Foundations of Retrieval-Augmented Generation (RAG)

1. Introduction and Architectural Motivation

Large Language Models (LLMs) have achieved remarkable natural language generation capabilities. However, they suffer from fundamental limitations including knowledge cutoffs, inability to access private corporate or domain-specific data, and severe hallucinations when reasoning over uncertain facts. Retrieval-Augmented Generation (RAG) resolves these issues by separating the external knowledge store from the parametric memory of the model.

In a standard RAG architecture, when a user poses a natural-language query, the system first retrieves semantically relevant passages from an external corpus using dense vector embeddings. These retrieved passages are then dynamically injected into the LLM context prompt alongside the original query. The language model is specifically instructed to synthesize its final answer grounded strictly on the supplied context.

2. Semantic Dense Vector Embeddings

Traditional keyword search methods such as Term Frequency-Inverse Document Frequency (TF-IDF) rely on exact lexical overlap. Consequently, they fail when users ask questions using synonyms or paraphrase concepts. Modern dense retrieval solves this by using pre-trained transformer encoders, such as all-MiniLM-L6-v2, to map text chunks into a shared 384-dimensional continuous vector space. In this latent space, semantically similar passages are positioned close to one another based on cosine similarity.

Document Chunking and Vector Indexing

3. Chunking Strategies and Granularity Trade-offs

Raw documents are typically too long to embed as a single vector without severe information dilution. Therefore, text segmentation or chunking is performed. Typical chunk sizes range between 500 and 800 words with an overlap of 50 to 100 words.

Sliding-window overlap is critical because it ensures that semantic continuity and cross-sentence dependencies are not severed at arbitrary chunk boundaries. Moreover, every individual chunk must preserve its provenance metadata, including source document name, page number, and chunk index, to enable reliable citations.

4. Vector Indexing with FAISS

Facebook AI Similarity Search (FAISS) is an open-source library optimized for high-performance dense vector similarity search. For exact nearest-neighbor search, FAISS provides IndexFlatIP (Inner Product) and IndexFlatL2 (Euclidean distance). When embeddings are L2-normalized to unit length, the inner product is mathematically identical to Cosine Similarity. Query latency with IndexFlatIP on CPU for small to medium corpora is sub-millisecond, providing scalable Top-K candidate retrieval.

Grounded LLM Generation & Evaluation Metrics

5. Grounded Prompting and Hallucination Mitigation

Sending retrieved context to the LLM requires strict prompt framing. The system prompt instructs the model: 'Answer the question using ONLY the provided context chunks. If the context does not contain sufficient facts, clearly state that the information is not available.'

Additionally, a relevance similarity threshold (such as 0.35 cosine similarity) filters out unrelated chunks before they reach the LLM, preventing the model from answering out-of-domain queries with ungrounded speculation.

6. Retrieval Evaluation Metrics (Precision@K, Recall@K, MRR)

To quantitatively benchmark retrieval quality, three core Information Retrieval metrics are evaluated:

1. Precision@K: The proportion of retrieved Top-K chunks that are truly relevant to the query.
2. Recall@K: The proportion of all known ground-truth relevant chunks that appear in the Top-K results.
3. Mean Reciprocal Rank (MRR): The average of reciprocal ranks ($1/\text{rank}$) of the first relevant chunk retrieved.

In empirical experiments, dense semantic retrieval consistently outperforms lexical TF-IDF on complex, synonym-heavy, and conceptual queries.